Project: Phylogenetic Analysis of Scorpion Toxins and Data Visualization of Hemolytic test

By Rima Jaber

Table of content

Project Overview

In modern bioinformatics, scientists analyze DNA sequences to understand evolutionary relationships between organisms. These analyses are often combined with experimental measurements to provide a more complete biological interpretation.

In this project, you will build a complete data analysis pipeline using Python. You will integrate biological metadata (concentration of crude venom and hemolysis%) with DNA sequence data to construct a phylogenetic tree and generate scientific visualizations.

Learning Objectives

By completing this project, you will learn how to:

- Work with biological datasets in CSV and FASTA formats
- Clean and preprocess data using Python
- Compute derived scientific values (hemolysis%)
- Compare DNA sequences using identity-based distance
- Construct a phylogenetic tree using the Neighbor Joining method
- Visualize biological data using different types of plots
- Interpret both experimental and evolutionary results

Project Data

CSV File (data.csv)

concofvenom	A	Aneg	Apos
0.013	1.85	0.15	1.95
0.027	1.42	0.15	1.95
0.067	0.88	0.15	1.95
0.093	0.45	0.15	1.95
0.13	0.22	0.15	1.95

FASTA File (projectsequences.fasta)

```
>AaH2_Androctonus_australis
VKDGYIVDDVNCTYFCGRNAYCNEECTKLKGESGYCQWASPYGNACYCYKLPDHVRTKGPGRCH

>Lqq5_Leiurus_quinquestriatus
VRDAYIAKNYNCVYECFRDAYCNELCTKNGASSGYCQWAGKYGNACWCYALPDNVPIRVPGKCH

>BjaIT_Buthus_judaicus
VRDAYIAKNYNCVYECFRNAYCNDLCTKNGAKSGYCQWAGKYGNACWCYALPDNVPIRVPGKCH

>Amm2_Androctonus_mauretanicus
VKDGYIVDDVNCTYFCGRNAYCNEECTKLKGESGYCQWASPYGNACYCYKLPDHVRTKGPGRCH
```

Project Tasks

Task 1: Load and Prepare Data

- Load the CSV file using pandas
- Clean column names
- Display the dataset

Task 2: Compute Hemolysis

- Calculate hemolysis% using the formula:

  100- (A-Aneg)/(Apos-Aneg)*100

- Add a new column named `hemolysis`
- Express the result in %

Task 3: Load DNA Sequences

- Read sequences from the FASTA file using Biopython
- Display sequence identifiers and sequences

Task 4: Create Alignment Object

- Build a Multiple Sequence Alignment object
- Prepare sequences for comparison

Task 5: Compute Distance Matrix

- Use identity-based distance with:

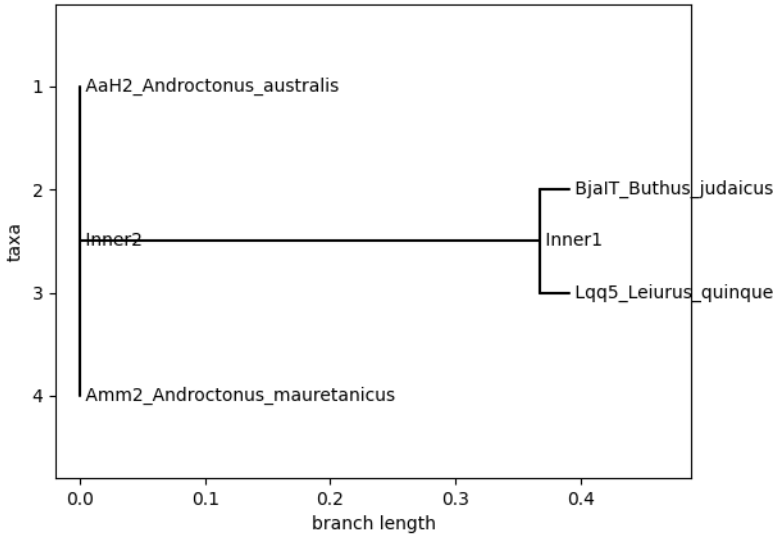
  DistanceCalculator("identity")

- Understand that distance = 1 − similarity

Task 6: Construct Phylogenetic Tree

- Use the Neighbor Joining method
- Display the tree in ASCII format
- Visualize the tree graphically

Phylogeny

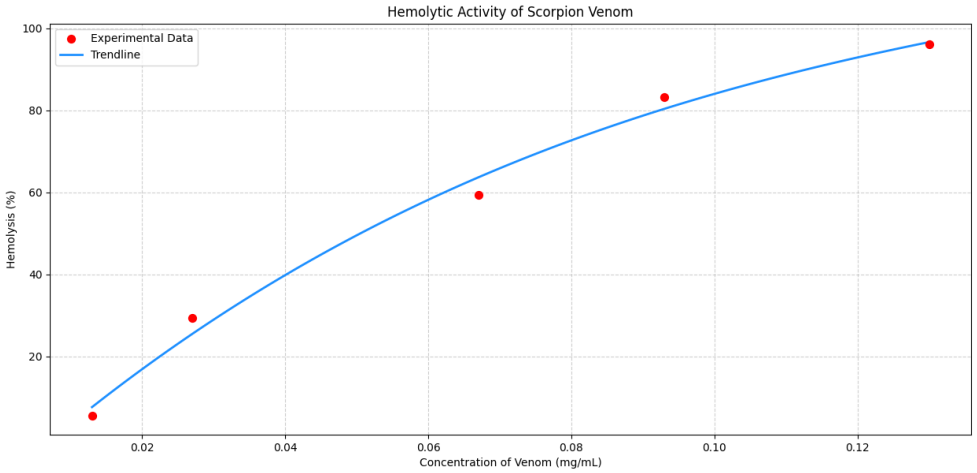


The data is split into two distinct families. Even though they are the same length (192 bp), their internal "letters" have evolved into two different versions of the same toxin.

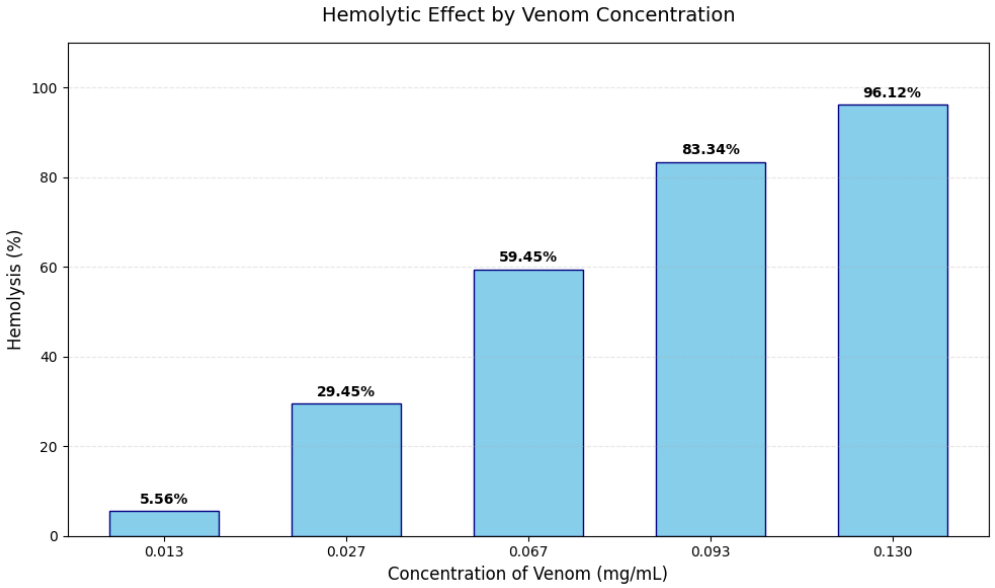
Task 7: Visualize Data

Curve plot

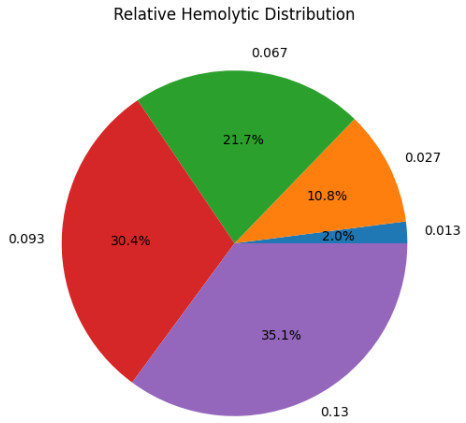
- Display hemolytic% vs concentrations of crude venom of *H.judaicus*


Hemolytic Activity of Scorpion Venom
Experimental Data
Trendline
Hemolysis (%)
Concentration of Venom (mg/mL)
100
80
60
40
20
0.02
0.04
0.06
0.08
0.10
0.12

Bar chart

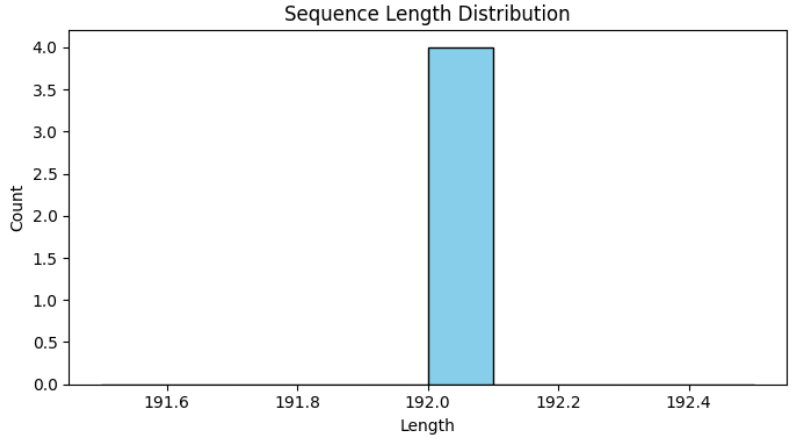

Hemolytic Effect by Venom Concentration
Hemolysis (%)
Concentration of Venom (mg/mL)
5.56%
29.45%
59.45%
83.34%
96.12%
0.013
0.027
0.067
0.093
0.130
0
20
40
60
80
100

Pie chart



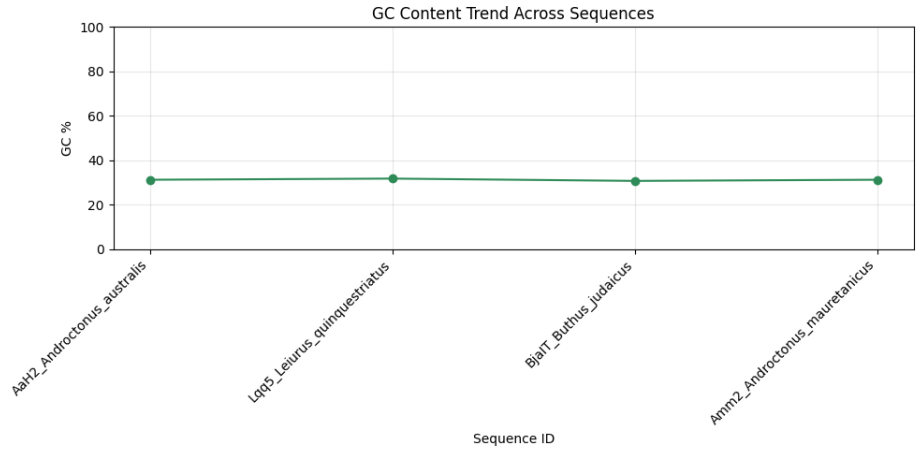
Pie charts are meant to show parts of a **whole** (like 100% of a budget or a population). In your experiment, the "total" (the whole pie) is just the sum of the hemolysis percentages from different tests, which doesn't actually represent a real physical thing. **Bar Chart** or **Dose-Response Curve** (the first two images) are much better for a lab report because they preserve the relationship between "how much venom" and "how much damage."

Sequence analysis



 Uniformity: The single, tall blue bar at the "192.0" mark indicates there is no variation in length across your dataset.

 Scorpion Toxin Biology: This is expected for these specific "long-chain" sodium channel toxins. They are highly conserved in evolution, meaning their gene structure remains very stable across different scorpion species like *Androctonus* and *Leiurus*.



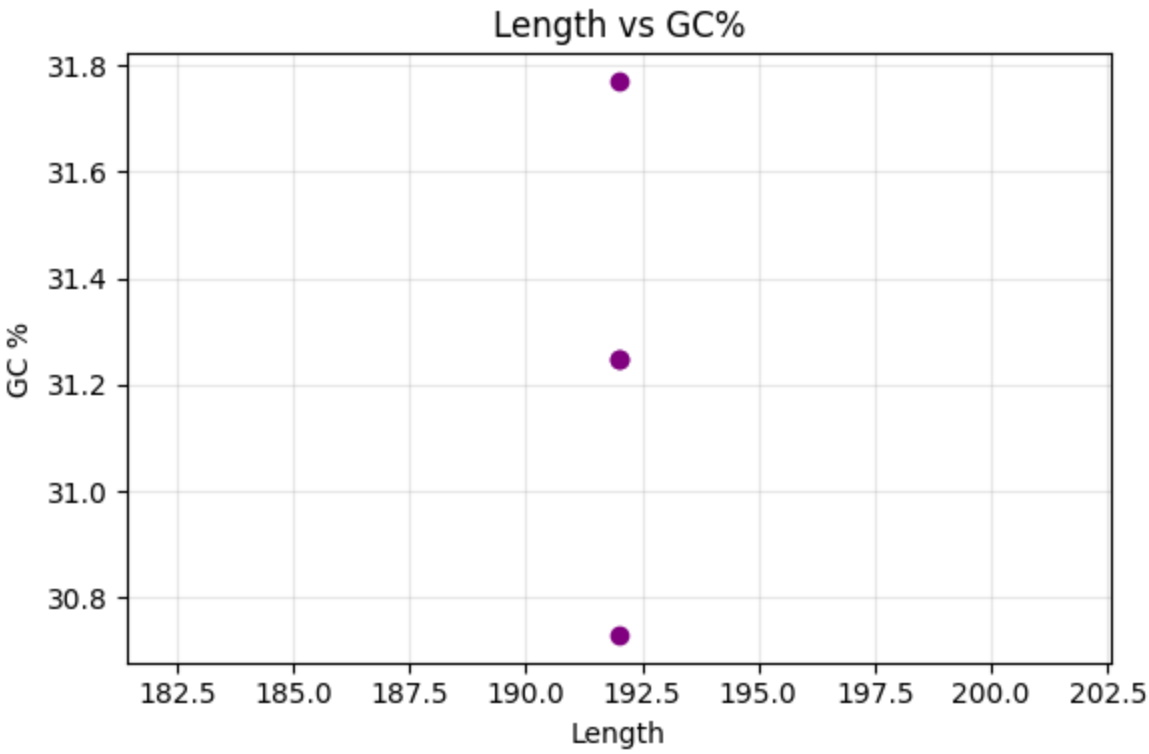
This graph shows that these scorpions toxins sequences are **AT-rich**, with a very stable GC content of around **31%**

 Low GC Bias: All sequences fall significantly below the 50% mark. This is a common feature in many scorpion venom genes, which tend to use more Adenine (A) and Thymine (T) than Guanine (G) and Cytosine (C).

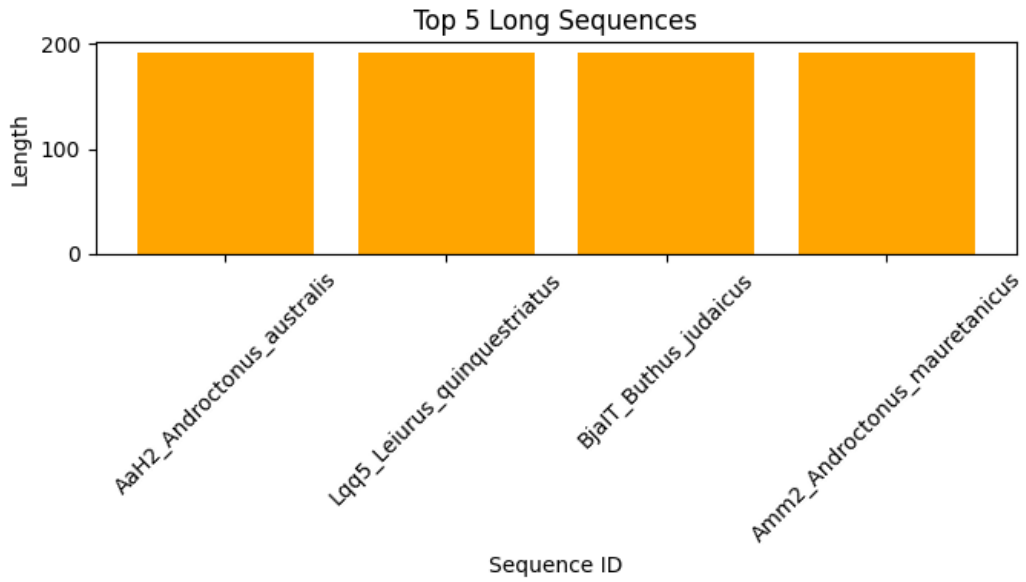
 High Conservation: The line is nearly flat, meaning the chemical "balance" of the DNA is almost identical across these different species (*Androctonus*, *Leiurus*, and *Buthus*).

 Slight Variations:

- **Lqq5** (*Leiurus quinquestriatus*) has the highest peak (approx. 31.77%).
- **BjaIT** (*Buthus judaicus*) shows a slight dip (approx. 30.73%).
- **AaH2** and **Amm2** are identical (31.25%), which makes sense as they are both from the *Androctonus* genus and are very closely related toxins.



- **Vertical Alignment:** All three dots line up perfectly at the **192** mark on the X-axis. This confirms that length is a "constant" in your current dataset—every sequence is identical in size.
- **Vertical Variance:** The spread of the dots along the Y-axis shows the internal differences. Even though they are the same size, the number of **G** and **C** bases varies slightly between the species.
- **Overlapping Points:** You provided 4 sequences, but there are only **3 dots**. This is because **AaH2** and **Amm2** both have a GC content of **31.25%**, so their dots are stacked directly on top of each other in the middle.



- **Perfect Consistency:** Since you provided four sequences and they are all exactly **192 bp**, the "Top 5" list simply displays all of them at the same height.
- **Biological Meaning:** In the world of scorpion toxins, these are "long-chain" peptides. The fact that they are all the same length despite coming from different genera (*Androctonus*, *Leiurus*, and *Buthus*) shows how **highly conserved** these toxins are. Evolution has kept the size identical because this specific length is likely required for the toxin to fold correctly and block sodium channels.

Sample Outputs

Metadata Output

```
=== METADATA ===
   concofvenom     a  aneg  apos
0        0.013  1.85  0.15  1.95
1        0.027  1.42  0.15  1.95
2        0.067  0.88  0.15  1.95
3        0.093  0.45  0.15  1.95
4        0.130  0.22  0.15  1.95
```

Concentration Output

```
=== Hemolysis ===
   concofvenom  hemolysis
```

```
0        0.013   5.555556
1        0.027  29.444444
2        0.067  59.444444
3        0.093  83.333333
4        0.130  96.111111
```

Distance Matrix (Example)

```
=== DISTANCE MATRIX ===
AaH2_Androctonus_australis  0.000000
Lqq5_Leiurus_quinquestriatus    0.390625    0.000000
BjaIT_Buthus_judaicus   0.390625    0.046875    0.000000
Amm2_Androctonus_mauretanicus   0.000000    0.390625    0.390625    0.000000
    AaH2_Androctonus_australis  Lqq5_Leiurus_quinquestriatus
BjaIT_Buthus_judaicus   Amm2_Androctonus_mauretanicus
```

Phylogenetic Tree (ASCII Example)

```
=== PHYLOGENETIC TREE (ASCII) ===
 , AaH2_Androctonus_australis
  |
  |                                                 __ BjaIT_Buthus_judaicus
_|_________________________________________________|
  |                                                 |__  Lqq5_Leiurus_quinquestriatus
  |
  | Amm2_Androctonus_mauretanicus
```

(The exact structure may vary slightly.)

```
=============== RESTART: C:/Users/star/Desktop/day3/scorpions.py ===============
```

Results:

	ID	Length	GC%	Motif_Count	Protein
0	AaH2_Androctonus_australis	192	31.25	11	VKDGYIVDDVNCTYFCGRNAYCNEECTKLKGESGYCQWASPYGNAC...
1	Lqq5_Leiurus_quinquestriatus	192	31.77	12	VRDAYIAKNYNCVYECFRDAYCNELCTKNGASSGYCQWAGKYGNAC...

2 BjaIT_Buthus_judaicus 192 30.73 12
VRDAYIAKNYNCVYECFRNAYCNDLCTKNGAKSGYCQWAGKYGNAC...

3 Amm2_Androctonus_mauretanicus 192 31.25 11
VKDGYIVDDVNCTYFCGRNAYCNEECTKLKGESGYCQWASPYGNAC...

📊 N50: 192

✅ Analysis complete. File saved: results.csv

Graph Outputs

- Bar chart showing hemolysis vs concentration of crude venom applied
- Curve showing also same variation

Analysis Questions

1. Based on the generated charts, does the scorpion venom exhibit a linear or a non-linear dose-response relationship?
2. From a data visualization perspective, why is a pie chart an inappropriate choice for representing independent venom concentrations?
3. Based on the maximum hemolysis observed (96.12%), is the 0.130 mg/mL dose considered a saturating concentration for this assay?
4. Based on the branching pattern at "Inner2," which two species share the most recent common ancestor and are considered sister taxa in this tree?
5. Comparing *Buthus judaicus* (BjaIT) and *Leiurus quinquestriatus* (Lqq5), how does their "Inner1" connection compare to the "Inner2" connection of the *Androctonus* species in terms of evolutionary time?

Code

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from Bio import SeqIO
from Bio.Align import MultipleSeqAlignment
from Bio.Phylo.TreeConstruction import DistanceCalculator, DistanceTreeConstructor
from Bio import Phylo

# ==============================
# 1. LOAD METADATA (CSV)
# ==============================

df = pd.read_csv("dataprep.csv")

# Clean ALL column names
df.columns = df.columns.str.strip().str.lower()

print("\n=== METADATA ===")
print(df)

# ======================
# 2. COMPUTE HEMOLYSIS
# ======================

df["hemolysis"] = df["hemolysis"]= 100-((df["a"] - df["aneg"])/(df["apos"] - df["aneg"])) * 100

print("\n=== Hemolysis ===")
```

```
print(df[["concofvenom", "hemolysis"]])

# ==============================
# 3. LOAD FASTA SEQUENCES
# ==============================

records = list(SeqIO.parse("toxins_sequences.fasta", "fasta"))

print("\n=== RAW SEQUENCES ===")
for r in records:
    print(r.id, r.seq)




# ==============================
# 4. MULTIPLE SEQUENCE ALIGNMENT
# ==============================

alignment = MultipleSeqAlignment(records)

# ==============================
#5. DISTANCE MATRIX (IDENTITY)
# ==============================

calculator = DistanceCalculator("identity")
dm = calculator.get_distance(alignment)

print("\n=== DISTANCE MATRIX ===")
print(dm)
```

```
# ==============================
# 6. PHYLOGENETIC TREE (NJ)
# ==============================

constructor = DistanceTreeConstructor()
tree = constructor.nj(dm)

print("\n=== PHYLOGENETIC TREE (ASCII) ===")
Phylo.draw_ascii(tree)

# ==============================
# 7. GRAPHICAL TREE
# ==============================

Phylo.draw(tree)




# ==============================
# 10. PIE CHART (HEMOLYSIS DISTRIBUTION)
# ==============================

plt.figure(figsize=(6, 6))
plt.pie(df["hemolysis"], labels=df["concofvenom"], autopct="%1.1f%%")

plt.title("Relative Hemolytic Distribution")
plt.show()
#=======================================================
```

```
concofvenom = np.array([0.013, 0.027, 0.067, 0.093, 0.130])
hemolysis = np.array([5.56, 29.45, 59.45, 83.34, 96.12])
plt.figure(figsize=(10, 6))
plt.scatter(concofvenom, hemolysis, color='red', s=50, label='Experimental Data', zorder=3)

z = np.polyfit(concofvenom, hemolysis, 3)
p = np.poly1d(z)
x_smooth = np.linspace(concofvenom.min(), concofvenom.max(), 100)

plt.plot(x_smooth, p(x_smooth), color='dodgerblue', linewidth=2, label='Trendline')

plt.title('Hemolytic Activity of Scorpion Venom')
plt.xlabel('Concentration of Venom (mg/mL)')
plt.ylabel('Hemolysis (%)')
plt.grid(True, linestyle='--', alpha=0.6)
plt.legend()

plt.tight_layout()
plt.show()
labels = ['0.013', '0.027', '0.067', '0.093', '0.130']
hemolysis = [5.56, 29.45, 59.45, 83.34, 96.12]

# 2. Create the figure
plt.figure(figsize=(10, 6))

# 3. Create the bars
# 'width' is set to 0.6 to ensure clear space between them
bars = plt.bar(labels, hemolysis, color='skyblue', edgecolor='navy', width=0.6)
```

```
# 4. Add data labels on top of each bar for clarity
for bar in bars:
    yval = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2, yval + 1, f'{yval}%',
             ha='center', va='bottom', fontsize=10, fontweight='bold')

# 5. Formatting
plt.title('Hemolytic Effect by Venom Concentration', fontsize=14, pad=15)
plt.xlabel('Concentration of Venom (mg/mL)', fontsize=12)
plt.ylabel('Hemolysis (%)', fontsize=12)
plt.ylim(0, 110)  # Extra space at the top for labels
plt.grid(axis='y', linestyle='--', alpha=0.3)

# 6. Final cleanup
plt.tight_layout()
plt.show()
Code to calculate GC content
from Bio import SeqIO
from Bio.Seq import Seq
import pandas as pd
import matplotlib.pyplot as plt

# ========================
# INPUT FILE
# ========================
fasta_file = "scorp.fasta"
motif = "ATG"

data = []
```

```
# ========================
# READ FASTA
# ========================
for record in SeqIO.parse(fasta_file, "fasta"):
    seq_id = record.id
    seq = str(record.seq).upper()

    length = len(seq)



    # GC content
    gc = (seq.count("G") + seq.count("C")) / length * 100

    # Protein translation
    protein = str(Seq(seq).translate(to_stop=True))

    # Motif count
    motif_count = seq.count(motif)

    data.append([seq_id, length, round(gc, 2), motif_count, protein])

# ========================
# DATAFRAME
# ========================
df = pd.DataFrame(data, columns=["ID", "Length", "GC%", "Motif_Count", "Protein"])

print("\n📄 Results:")
print(df)
```

```
# ========================
# N50 FUNCTION
# ========================
def calculate_n50(lengths):
    lengths = sorted(lengths, reverse=True)
    total = sum(lengths)
    cum = 0

    for l in lengths:
        cum += l
        if cum >= total / 2:
            return l

n50 = calculate_n50(df["Length"])
print("\n📊 N50:", n50)

# ========================
# SAVE RESULTS
# ========================
df.to_csv("results.csv", index=False)

# ========================================
# 📊 1. LENGTH DISTRIBUTION
# ========================================
plt.figure(figsize=(7,4))
plt.hist(df["Length"], bins=10, color="skyblue", edgecolor="black")
plt.title("Sequence Length Distribution")
plt.xlabel("Length")
```

```
plt.ylabel("Count")
plt.tight_layout()
plt.show()

# ========================================
# 📊 2. GC% TREND LINE (BEST CHOICE)
# ========================================
plt.figure(figsize=(10,5))

plt.plot(
    df["ID"],
    df["GC%"],
    marker="o",
    linestyle="-",
    color="seagreen"
)

plt.title("GC Content Trend Across Sequences")
plt.xlabel("Sequence ID")
plt.ylabel("GC %")

plt.xticks(rotation=45, ha="right")
plt.ylim(0, 100)
plt.grid(alpha=0.3)

plt.tight_layout()
plt.show()

# ========================================
```

```
# 📊 3. LENGTH vs GC
# ========================================
plt.figure(figsize=(6,4))
plt.scatter(df["Length"], df["GC%"], color="purple")
plt.title("Length vs GC%")
plt.xlabel("Length")
plt.ylabel("GC %")
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()

# ========================================
# 📊 4. TOP SEQUENCES BY LENGTH
# ========================================
top = df.sort_values(by="Length", ascending=False).head(5)

plt.figure(figsize=(7,4))
plt.bar(top["ID"], top["Length"], color="orange")
plt.title("Top 5 Long Sequences")
plt.xlabel("Sequence ID")
plt.ylabel("Length")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

print("\n✅ Analysis complete. File saved: results.csv")
```